

Implement a solution for a constraint satisfaction problem using branch and bound/Backtracking for: A) n-queens problem B) Graph coloring problem

Code :-

N=4

b=[[0]*N for _ in range(N)]

def f(r):

if r==N: [print(i) for i in b]; print(); return

for c in range(N):

if all(b[i][c]==0 for i in range(r)) and \

all(b[r-i-1][c-i-1]==0 for i in range(min(r,c))) and \

all(b[r-i-1][c+i+1]==0 for i in range(min(r,N-c-1))):

b[r][c]=1; f(r+1); b[r][c]=0

f(0)

g={0:[1,2],1:[0,2],2:[0,1]}

c={}

def f(v):

if v==len(g):

print(c)

return

for i in ["Red","Green","Blue"]:

if all(c.get(n)!=i for n in g[v]):

c[v]=i

f(v+1)

del c[v]

f(0)

Theory :-

AIM

To solve:

- 1. N-Queens Problem**
- 2. Graph Coloring Problem using Backtracking / Branch and Bound technique.**

THEORY

Constraint Satisfaction Problem (CSP)

A Constraint Satisfaction Problem consists of:

- **Variables**
- **Domains (possible values)**
- **Constraints**

The goal is to assign values to variables without violating constraints.

Backtracking is a recursive method used to solve CSPs by trying possible solutions and rejecting invalid ones.

A) N-Queens Problem

Theory

The N-Queens problem is placing N queens on an $N \times N$ chessboard so that:

- No two queens are in the same row
- Same column
- Same diagonal

Backtracking places queens row by row and checks whether the position is safe.

Algorithm Steps – N Queens

1. Start
2. Place queen in first row
3. Check if position is safe:
 - No queen in same column ○ No queen in same diagonal
4. If safe, place queen
5. Recursively place queen in next row
6. If no position is safe, backtrack
7. Repeat until all queens are placed
8. Print solution
9. Stop

B) Graph Coloring Problem

Theory

Graph coloring means assigning colors to graph vertices such that:

- No two adjacent vertices have the same color

Backtracking tries different colors for each vertex and checks constraints.

Algorithm Steps – Graph Coloring

1. Start
2. Assign first color to first vertex

3. Check whether adjacent vertices have same color
4. If safe, assign color
5. Move to next vertex recursively
6. If no color is possible, backtrack
7. Repeat until all vertices are colored
8. Print solution
9. Stop